

Memoria Técnica de Desarrollo: KRAsky ERP

1. Introducción y Alcance

El proyecto **KRAsky ERP** consiste en el desarrollo de una aplicación web de gestión empresarial (ERP) utilizando el ecosistema Java Spring Boot. El objetivo principal de esta primera fase ha sido establecer una arquitectura robusta, escalable y segura, centrada en la gestión de usuarios, roles y autenticación persistente contra base de datos.

Tras esa fase inicial, el proyecto evolucionó hacia un sistema ERP completo para una aerolínea, abarcando la gestión de la flota de aviones, la programación de vuelos, el CRM de pasajeros y un motor transaccional de reservas y emisión de billetes.

2. Stack Tecnológico

Para el desarrollo se han seleccionado tecnologías estándar de la industria que garantizan estabilidad y facilidad de mantenimiento:

- **Lenguaje:** Java 17 (LTS).
- **Framework Backend:** Spring Boot 3.2.3.
- **Gestión de Dependencias:** Maven.
- **Base de Datos:** MySQL 8.0.
- **ORM (Mapeo Objeto-Relacional):** Hibernate / Spring Data JPA.
- **Motor de Plantillas (Frontend):** Thymeleaf.
- **Diseño (UI):** Bootstrap 5.3.
- **Seguridad:** Spring Security 6.
- **Herramientas Auxiliares:** Lombok (reducción de código repetitivo), Spring DevTools.

3. Configuración e Inicialización del Proyecto

3.1. Estructura del Proyecto (**pom.xml**)

Se configuraron las dependencias esenciales en el archivo **pom.xml**:

- **spring-boot-starter-web:** Para el servidor Tomcat embebido y arquitectura MVC.
- **spring-boot-starter-data-jpa:** Para la conexión a BBDD.
- **spring-boot-starter-security:** Para el control de acceso.
- **mysql-connector-j:** Driver JDBC.

- **lombok**: Para automatizar Getters, Setters y Constructores.

3.2. Conexión a Base de Datos (**application.properties**)

Se estableció la conexión con MySQL definiendo las propiedades de la fuente de datos (DataSource) y configurando Hibernate para la actualización automática del esquema (**ddl-auto=update**) durante el desarrollo.

4. Arquitectura de Software (Modelo MVC)

El proyecto sigue una arquitectura en capas para separar responsabilidades:

4.1. Capa de Modelo (Entidades)

Se diseñaron las entidades que representan las tablas en la base de datos:

- **Usuario.java**: Entidad principal. Contiene atributos como **username**, **password**, **email**, **enabled**. Utiliza anotaciones JPA (**@Entity**, **@Table**) y Lombok (**@Data**).
- **Ro1.java**: Entidad para gestionar permisos (**ROLE_USER**, **ROLE_ADMIN**).
- **Relación**: Se implementó una relación **Many-to-Many** entre Usuarios y Roles (**@ManyToMany**), permitiendo una gestión flexible de permisos.
- **Avion.java**: Gestión de flota con matrículas únicas y capacidades por clase.
- **Pasajero.java**: Información de clientes con validación de DNI y Email únicos.
- **Vuelo.java**: Programación de rutas. Incluye relaciones **@ManyToOne** con Avión y uso de Enums para estados (**PROGRAMADO**, **CANCELADO**, etc.).
- **Reserva.java**: Entidad transaccional que vincula Vuelo y Pasajero, generando códigos de reserva únicos y calculando precios.

4.2. Capa de Repositorio (Acceso a Datos)

Se crearon interfaces que extienden de **JpaRepository**. Esto permite realizar operaciones CRUD (Create, Read, Update, Delete) sin escribir SQL manual.

- **UsuarioRepository**: Incluye el método personalizado **Optional<Usuario> findByUsername(String username)** fundamental para el proceso de login.

4.3. Capa de Servicio (Lógica de Negocio)

Esta capa actúa de intermediario entre los controladores y los repositorios, encapsulando la lógica de la aplicación. (Principalmente utilizada en la lógica de seguridad detallada en la sección 5).

Se implementaron reglas de negocio complejas, como impedir reservas en vuelos llenos, validar que la fecha de llegada sea posterior a la de salida, o evitar duplicidad de DNIs y matrículas mediante excepciones personalizadas (**BusinessException**).

4.4. Capa de Vista (Frontend)

Se desarrollaron plantillas HTML5 utilizando **Thymeleaf** para renderizar datos dinámicos del servidor.

- **login.html**: Formulario de acceso estilizado con Bootstrap. Incluye manejo de errores (**param.error**) y mensajes de cierre de sesión.

5. Implementación del Módulo de Seguridad

Esta fue la fase más crítica y compleja del desarrollo. El objetivo era abandonar la autenticación en memoria y validar usuarios reales almacenados en MySQL.

5.1. Configuración de Seguridad (**SecurityConfig.java**)

Se definió la clase de configuración para controlar el flujo HTTP:

- **Encriptación**: Implementación de **BCryptPasswordEncoder** para asegurar que las contraseñas nunca se guarden en texto plano.
- **FilterChain**: Definición de rutas públicas (**/login**, recursos estáticos) y privadas (resto de la aplicación).
- **AuthenticationProvider**: Configuración del **DaoAuthenticationProvider** para conectar el sistema de login con nuestro servicio de base de datos.

5.2. Servicio de Detalles de Usuario (**UserDetailsServiceImpl.java**)

Se implementó la interfaz **UserDetailsService** de Spring.

- **Función**: Este servicio intercepta el login, busca al usuario en la BBDD usando el Repositorio, y convierte la entidad **Usuario** en un objeto **UserDetails** compatible con Spring Security.
- **Manejo de Roles**: Convierte la lista de roles del usuario en **Authorities**.

6. Arquitectura Híbrida y API REST

Para cumplir con los requisitos de interoperabilidad, el sistema se diseñó con una arquitectura híbrida que sirve tanto contenido HTML (para la administración web) como datos JSON (para integraciones externas).

6.1. Patrón DTO

Se implementó un patrón de diseño DTO para desacoplar la capa de persistencia de la capa de presentación.

- **Objetivo**: Evitar bucles infinitos en las respuestas JSON (problema de referencia circular en relaciones bidireccionales) y ocultar datos sensibles de las entidades.

- **Implementación:** Clases como `VueloDTO` o `ReservaDTO` que transportan solo los datos necesarios entre el cliente y el servidor.

6.2. Controladores REST

Se desarrolló un paquete específico `controller.rest` con endpoints para la gestión externa.

- **Métodos GET:** Búsquedas avanzadas (por origen/destino, vuelos disponibles, reservas por pasajero).
- **Métodos POST/PUT:** Creación y edición con validación estricta de datos (`@Valid`).
- **Respuestas:** Uso de `ResponseEntity` para controlar los códigos de estado HTTP (200 OK, 404 Not Found, 400 Bad Request).

7. Gestión de Errores y Robustez

Se implementó un sistema centralizado de manejo de excepciones para mejorar la experiencia de usuario y la seguridad de la API.

7.1. GlobalExceptionHandler

Uso de la anotación `@RestControllerAdvice` para capturar errores en toda la aplicación:

- **ResourceNotFoundException:** Devuelve 404 si se busca un ID inexistente.
- **BusinessException:** Devuelve 400 si se violan reglas de negocio (ej: intentar borrar un avión con vuelos asignados).
- **Validación:** Captura errores de formato (precios negativos, fechas pasadas) y devuelve un JSON limpio indicando qué campo falló.

7.2. Validaciones en la Capa Web

Integración de `BindingResult` en los controladores Thymeleaf para mostrar errores visuales en los formularios (marcos rojos y mensajes de alerta) sin perder los datos introducidos por el usuario.

8. Desafíos Técnicos y Resolución de Problemas (Troubleshooting)

Durante la integración final, se resolvieron tres incidencias críticas que impedían el funcionamiento del sistema:

8.1. Conflicto de Nombres en Lombok (Booleanos)

- **Problema:** El código fallaba al compilar o al ejecutar porque Spring Security no encontraba el método `getEnabled()`.

- **Causa:** La librería Lombok genera los "getters" de campos primitivos `boolean` con el prefijo `is` (ej. `isEnabled()`) en lugar de `get`.
- **Solución:** Se corrigió la llamada en `UserDetailsServiceImpl` para usar el método correcto `usuario.isEnabled()`.

8.2. Truncamiento de Hashes en Base de Datos

- **Problema:** Error persistente de "Bad Credentials" o "Encoded password does not look like BCrypt", a pesar de que la contraseña parecía correcta.
- **Análisis:** Se detectó que la columna `password` en MySQL tenía una longitud por defecto de 50 caracteres (`VARCHAR(50)`). El algoritmo BCrypt genera hashes de 60 caracteres.
- **Consecuencia:** Al guardar usuarios, MySQL cortaba los últimos 10 caracteres del hash, corrompiendo la contraseña.

Solución: Se ejecutó una migración de base de datos:

SQL

```
ALTER TABLE usuarios MODIFY COLUMN password VARCHAR(255);
```

8.3. Inconsistencia de Datos de Prueba

- **Problema:** Tras arreglar la base de datos, el login seguía fallando con ciertos usuarios.
- **Causa:** Existían usuarios creados *antes* de la ampliación de la columna (con contraseñas cortadas) y usuarios creados *después* (correctos).
- **Solución:** Se realizó una limpieza total de usuarios de prueba y se insertó un usuario administrador limpio y verificado mediante scripts SQL.

8.4. Mapeo de Enums y Base de Datos

- **Problema:** Error `ConverterNotFoundException` al intentar guardar estados de vuelo o clases de asiento.
- **Causa:** JPA intentaba guardar objetos complejos Java (Enums) en columnas de texto plano (VARCHAR) sin instrucciones claras.
- **Solución:** Uso de `@Enumerated(EnumType.STRING)` en las entidades y métodos de conversión manual (`valueOf()` y `name()`) en los servicios para transformar los Strings de los DTOs en Enums de lógica de negocio.

8.5 Conflicto de Rutas de API REST

- **Problema:** Al hacer una petición GET con parámetros (`/api/vuelos?origen=X`), el sistema devolvía todos los vuelos ignorando el filtro.
- **Causa:** Conflicto entre el endpoint raíz (`findAll`) y el de búsqueda. Spring no discriminaba correctamente solo por parámetros.

- **Solución:** Se definieron rutas explícitas (`/api/vuelos/buscar` y `/api/vuelos/disponibles`) para segregar claramente las responsabilidades de cada endpoint.

9. Conclusión y Estado Actual

El proyecto KRAskY ERP ha evolucionado desde un sistema básico de autenticación hasta convertirse en una plataforma de gestión integral plenamente funcional. Se han cumplido el 100% de los requisitos funcionales y técnicos:

1. **Arquitectura:** Separación estricta en capas (Controller, Service, Repository, Entity/DTO).
2. **Seguridad:** Login persistente y encriptación robusta.
3. **Funcionalidad:** Gestión completa (CRUD) de flota, programación de vuelos y motor de reservas.
4. **Calidad:** Sistema tolerante a fallos mediante control de excepciones global y validaciones de negocio.

El resultado es una aplicación escalable, lista para despliegue y con una API documentada para futuras integraciones móviles o web externas.